МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом. Вариант 2р.

Студент гр. 4345 Бабий Д.Н.
Преподаватель Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы:

Изучить методы рекурсивного бэктрекинга для разбиения квадратной доски на минимальное число меньших квадратов и исследовать, как время выполнения алгоритма зависит от размера квадрата.

Постановка задачи

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до N −1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.

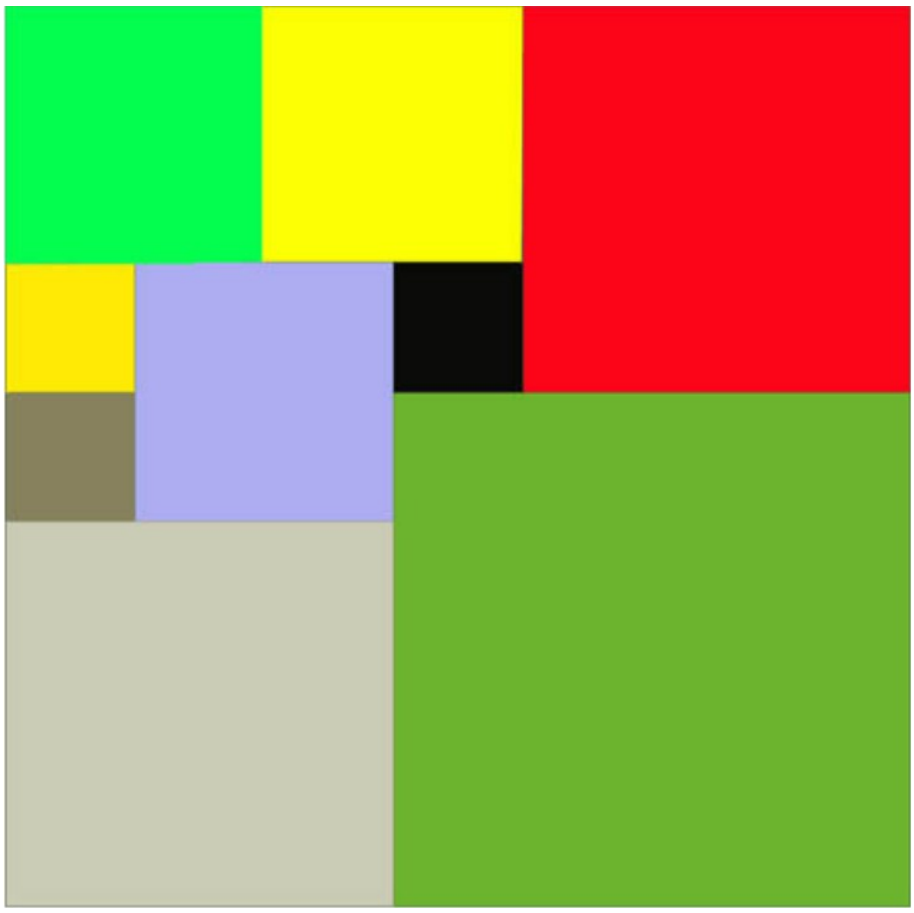

Рис. 1 – Пример решения

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число

N (2≤N≤20).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \le x,y \le N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Выполнение работы

Основные оптимизации

1)Хранение доски в виде списка списков из нулей слишком дорого, поэтому поле реализовано как массив целых чисел board длины N, где каждый бит числа соответствует клетке 1×1: 1 – клетка занята, 0 – клетка свободна. Такое представление позволяет проверять возможность размещения квадрата за O(1) на строку.

2)Применяется масштабирование: если число составное, выгоднее и быстрее разбивать квадрат со стороной, равной наименьшему делителю.

3)Используется предзаполнение симметричными квадратами и backtrack, начиная с максимально больших квадратов, что сокращает число вариантов и ускоряет поиск оптимального разбиения.

Пошаговое описание работы

Вначале проходит проверка длины квадрата: если число составное, то применяется масштабирование, фактически уменьшая квадрат. Если простое, то ничего не происходит.

Можно выделить три случая работы алгоритма:

1) N — четное

Если сторона квадрата четная, то минимальное количество квадратов разбиения всегда одинаковое — 4.

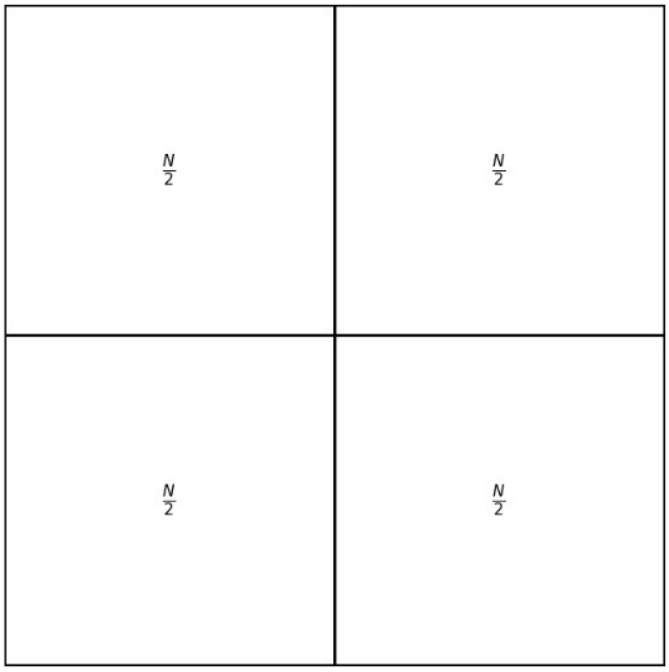

Рисунок 1. Разбиение квадрата N четное

2) N — нечетное и составное

Здесь разбиение всегда разнится и зависит от делителей числа

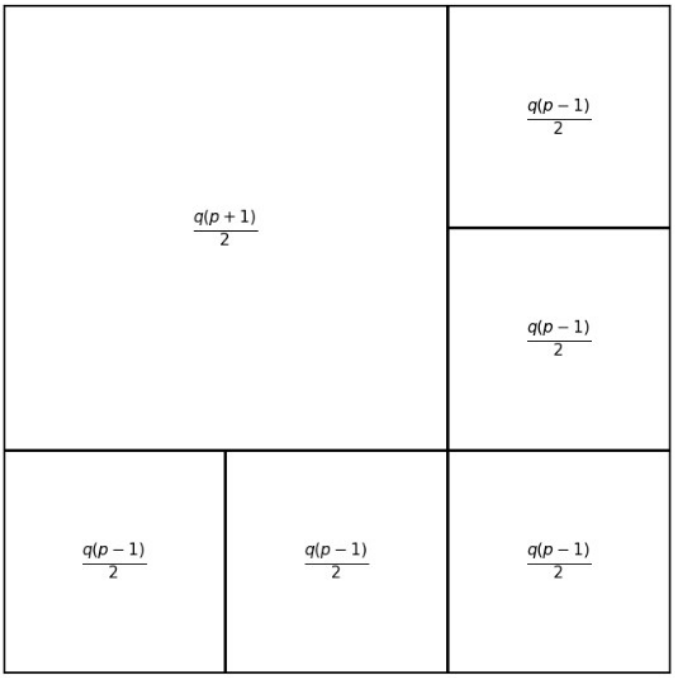

Рисунок 2. Пример разбиение при N=9

где p,q — делители числа N(p — меньший делитель).

Первым шагом мы масштабируем квадрат, то есть его длина становится p. Далее строим самый большой квадрат в левом верхнем углу размера (p + 1)/2 и два квадрата по его сторонам размера (p - 1)/2.

Далее возвращаем бывший размер(умножаем на q).

3) N — простое

Самый дорогой и долгий случай. В нем масштабирование невозможно, как и в предыдущем случае строим самый большой квадрат в левом верхнем углу размера (N + 1)/2 и два квадрата по его сторонам размера (N- 1)/2. Далее с помощью бэктрекинга рекурсивно заполняем оставшиеся место.

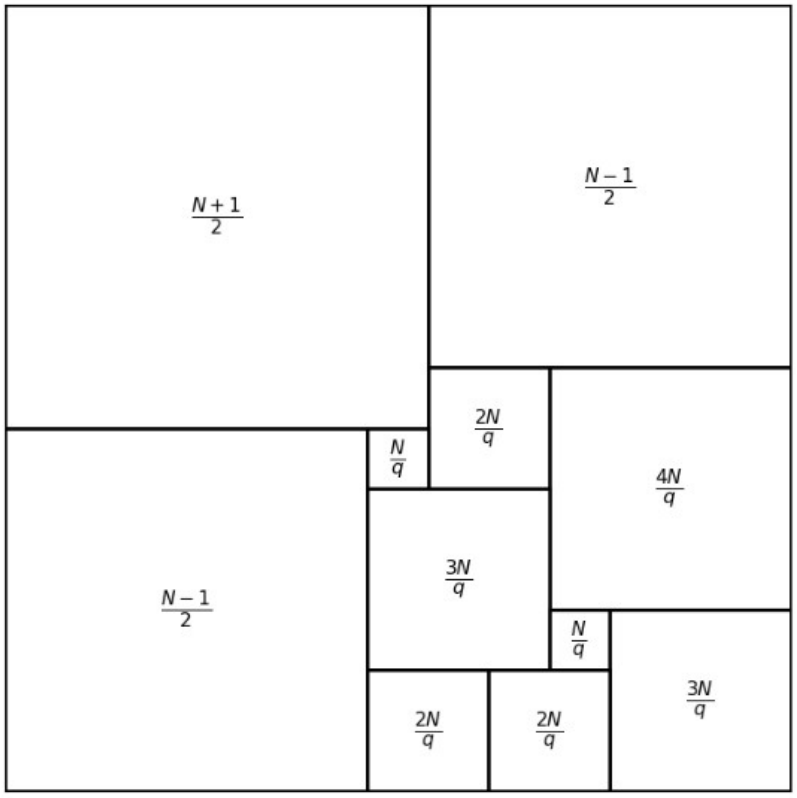

Рисунок 3. Разбиение квадрата N неправильное(N=13)

После предварительного размещения трёх квадратов оставшаяся часть поля заполняется с помощью рекурсивного поиска с возвратом (backtracking). Алгоритм находит первую свободную клетку и определяет максимальный размер квадрата, который можно разместить

в этой позиции. Затем последовательно пробует разместить квадраты всех допустимых размеров — от наибольшего к наименьшему. Если текущее количество квадратов становится больше или равно уже найденному минимуму, ветка поиска отсекается. При полном заполнении поля решение сохраняется как лучшее, если оно использует меньше квадратов.

Используемые функции

1) def scale_down():

Функция факторизации числа. Итерируется по всем числам от 2 до n + 1, break при нахождении делителя. Используется для поиска составных и простых чисел.

Аргументы:

- n: int — размер стороны квадрата

Возвращает:

- n: int — обновленное n, которая учитывает масштабирование
- i: int — коэффициент масштабирования

2) def find_first()

Функция поиска первой свободной клетки на поле. Последовательно проверяет строки массива board. Если строка заполнена не полностью, с помощью побитовых операций находится первый нулевой бит, соответствующий свободной клетке.

Возвращает:

- x, y: tuple[int, int] — координаты первой свободной клетки
- None — если свободных клеток не остал

3) def place()

Функция размещения или удаления квадрата на поле. С помощью битовой маски изменяет состояние клеток в массиве board. При

размещении биты устанавливаются в 1, при удалении — сбрасываются в 0.

Аргументы:

- x: int — координата по горизонтали
- y: int — координата по вертикали
- size: int — длина стороны квадрата
- add: bool — флаг добавления или удаления квадрата

4) def max_square()

Функция определения максимального размера квадрата, который можно разместить в позиции (x, y) без пересечения с уже занятыми клетками. Проверка выполняется с использованием битовых масок.

Аргументы:

- x: int — координата по горизонтали
- y: int — координата по вертикали

Возвращает:

- size: int — максимальный допустимый размер квадрата

5) def prefill()

Функция предварительного заполнения поля несколькими крупными квадратами перед запуском основного рекурсивного алгоритма. Её задача — уменьшить размер области поиска и сократить количество вариантов, которые должен проверить алгоритм backtracking.

- mid = (n + 1) // 2 — размер большого квадрата в верхнем левом углу.
- small = n // 2 — размер двух меньших квадратов

6) def backtrack()

Рекурсивная функция поиска решения методом перебора с возвратом (backtracking). Находит первую свободную клетку, определяет максимальный размер квадрата, который можно разместить в этой позиции, и последовательно пробует все возможные размеры. При достижении полного заполнения поля обновляет лучшее найденное решение.

Аргументы:

- count: int — текущее количество размещённых квадратов

Оценка сложности алгоритма

1) Для случая, когда N — чётное число, сложность алгоритма всегда примерно равна O(1) (происходит также единственная итерация от N/2 до 0, чтобы заполнить последний квадрат).

2) Для случая, когда N — составное нечётное число, В этом случае применяется масштабирование: исходная задача сводится к доске размера p, где p — наименьший делитель числа N (из-за масштабирования).

После этого используется рекурсивный алгоритм поиска с возвратом, который в общем случае имеет экспоненциальную сложность. Таким образом, теоретическая оценка времени работы имеет вид:

O(c^p), где c — некая константа.

Предварительное размещение крупных симметричных квадратов, а также отсечение неперспективных ветвей существенно уменьшают количество перебираемых вариантов. Поэтому на практике алгоритм работает значительно быстрее и для небольших значений N может демонстрировать близкое к константному время выполнения, однако в худшем случае сложность остаётся экспоненциальной.

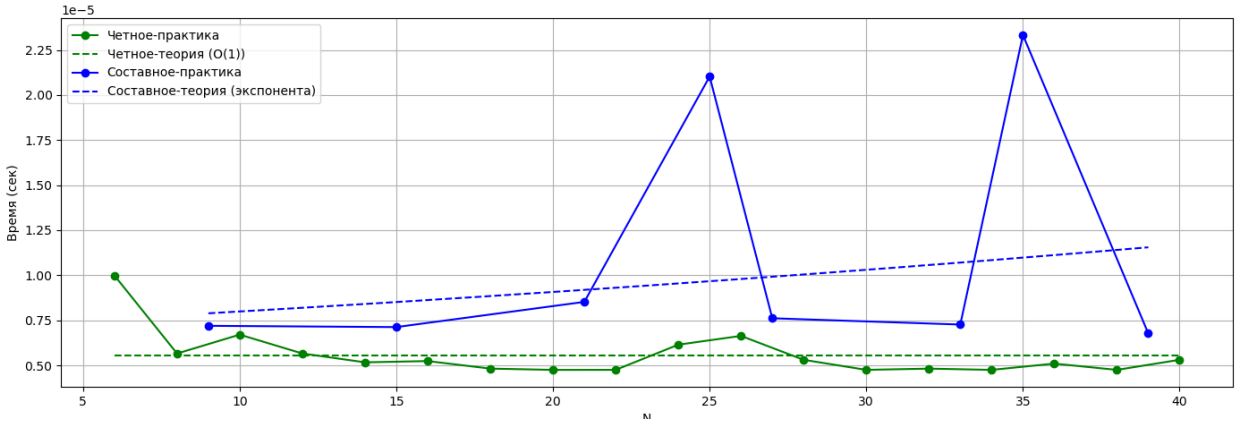

Рисунок 4. Два графика. N четное, N составное и нечетное

3) Для простого числа N наименьший делитель равен самому числу, поэтому масштабирование не применяется, и алгоритм работает с полной доской размера N×N. В этом случае используется рекурсивный перебор с возвратом, при котором на каждом шаге рассматриваются различные варианты размещения квадратов. Число таких вариантов растёт экспоненциально с увеличением размера доски.

Таким образом, теоретическая оценка сложности имеет вид:

O(c^N), где c — некая константа

Предварительное размещение как и в 2-ом случае ускоряет работу. Однако в худшем случае алгоритм остаётся экспоненциальным по времени выполнения.

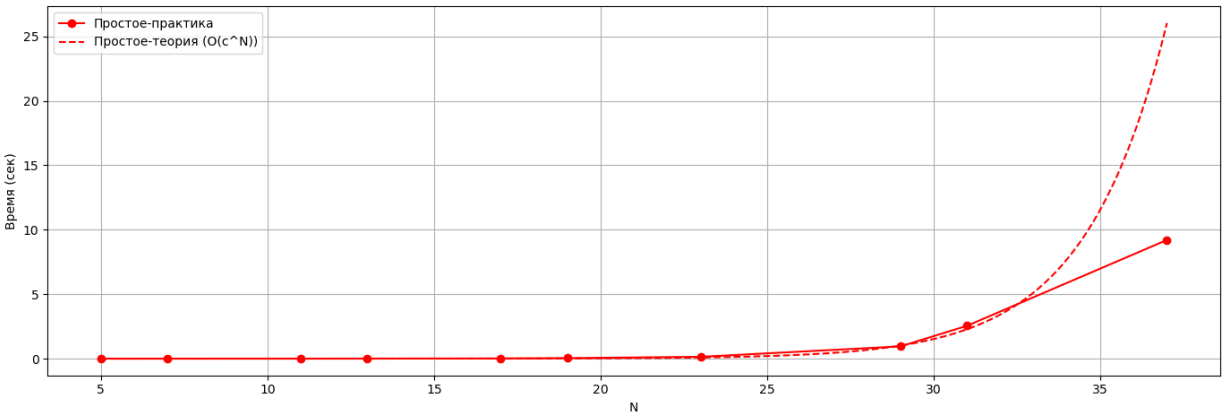

Рисунок 5. График N простое

Исходный код программы см. в приложении А.

№	Входные данные	Выходные данные	Комментарий
1	17	12 1 1 9 10 1 8 1 10 8 10 9 2 12 9 4 16 9 2 9 10 1 9 11 3 16 11 2 12 13 1 13 13 5 9 14 4	Введено простое число. Масштабирование не применилось
2	49	49 9 1 1 28 29 1 21 1 29 21 29 22 14 43 22 7 22 29 7 43 29 7 22 36 14 36 36 14	Составное и нечетное, масштабирование уменьшило доску в 7 раз.
3	44	4	Четное, поистроилось 4

		1 1 22 23 1 22 1 23 22 23 23 22	квадрата

Таблица 1 – Результаты тестирования

Выводы

Программа реализует поиск с возвратом для разбиения квадратной доски на минимальное число меньших квадратов. Алгоритм перебирает все варианты, начиная с максимально больших квадратов, что позволяет быстро сужать область поиска. Для чётных и большинства составных чисел алгоритм почти константный, а для простых растёт экспоненциально. Использование крупных квадратов и отсечения неперспективных ветвей делает метод эффективным для досок среднего размера и гарантирует нахождение оптимального решения.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
import time
import matplotlib.pyplot as plt
import numpy as np



def solve(n_input, silent=True):
    def get_scale(val):
        for i in range(2, val + 1):
            if val % i == 0:
                return i, val // i
        return val, 1

    n_local, scale = get_scale(n_input)
    board = [0] * n_local
    full = (1 << n_local) - 1
    state = {"best_k": 16, "best_res": []}
    current_res = []

    def place(x, y, size, add=True):
        mask = ((1 << size) - 1) << x
        for i in range(size):
            if add:
                board[y + i] |= mask
            else:
                board[y + i] &= ~mask

    def find_first():
        for y in range(n_local):
            if board[y] != full:
                free = ~board[y] & full
                x = (free & -free).bit_length() - 1
                return x, y
```

```
    return None

def max_sq(x, y):
    size = 1
    while x + size < n_local and y + size < n_local:
        mask = ((1 << (size + 1)) - 1) << x
        if any(board[y + i] & mask for i in range(size + 1)):
            break
        size += 1
    return size

def backtrack(count):
    if count >= state["best_k"]:
        return
    pos = find_first()
    if pos is None:
        state["best_k"] = count
        state["best_res"] = current_res[:]
        return
    x, y = pos
    m = max_sq(x, y)
    for size in range(m, 0, -1):
        place(x, y, size, True)
        current_res.append((x, y, size))

        backtrack(count + 1)

        current_res.pop()
        place(x, y, size, False)

def prefill():
    mid, small = (n_local + 1) // 2, n_local // 2

    place(0, 0, mid)
    current_res.append((0, 0, mid))

    place(mid, 0, small)
```

```
        current_res.append((mid, 0, small))

        place(0, mid, small)
        current_res.append((0, mid, small))

    prefill()
    backtrack(3)

    if not silent:
        print(state["best_k"])
        for x, y, size in state["best_res"]:
            print(f"{x * scale + 1} {y * scale + 1} {size * scale}")

    return state["best_k"]



def run_benchmarks():

    even_ns = np.array([n for n in range(6, 41, 2)])
    comp_ns = np.array(
        [
            n
            for n in range(6, 41)
             if any(n % i == 0 for i in range(2, int(n**0.5) + 1)) and
n % 2 != 0
        ]
    )
    prime_ns = np.array([5, 7, 11, 13, 17, 19, 23, 29, 31, 37])

    def measure_times(ns, repeats=7, warmup=4):
        times = []
        for n in ns:

            run_times = []
            for _ in range(repeats):
                start = time.perf_counter()
                solve(n, silent=True)
```

```
            run_times.append(time.perf_counter() - start)

        times.append(np.median(run_times))

    return np.array(times)

even_times = measure_times(even_ns)
comp_times = measure_times(comp_ns)
prime_times = measure_times(prime_ns)

coef_even = np.mean(even_times)
theor_even = np.full_like(even_ns, coef_even, dtype=float)

log_c_times = np.log(comp_times)
alpha_c, beta_c = np.polyfit(comp_ns, log_c_times, 1)
x_comp_smooth = np.linspace(min(comp_ns), max(comp_ns), 200)
theor_comp = np.exp(alpha_c * x_comp_smooth + beta_c)

log_p_times = np.log(prime_times)
alpha_p, beta_p = np.polyfit(prime_ns, log_p_times, 1)
x_prime_smooth = np.linspace(min(prime_ns), max(prime_ns), 200)
theor_prime = np.exp(alpha_p * x_prime_smooth + beta_p)

plt.figure(figsize=(14, 10))

ax1 = plt.subplot(2, 1, 1)
ax1.plot(even_ns, even_times, "g-o", label="Четное-практика")
ax1.plot(even_ns, theor_even, "g--", label="Четное-теория (O(1))")

ax1.plot(comp_ns, comp_times, "b-o", label="Составное-практика")
ax1.plot(
    x_comp_smooth,
    theor_comp,
    "b--",
    label="Составное-теория (экспонента)",
)
```

```
    ax1.set_xlabel("N")
    ax1.set_ylabel("Время (сек)")
    ax1.grid(True)
    ax1.legend()

    ax2 = plt.subplot(2, 1, 2)
    ax2.plot(prime_ns, prime_times, "r-o", label="Простое-практика")
    ax2.plot(
        x_prime_smooth,
        theor_prime,
        "r--",
        label="Простое-теория (O(c^N))",
    )

    ax2.set_xlabel("N")
    ax2.set_ylabel("Время (сек)")
    ax2.grid(True)
    ax2.legend()

    plt.tight_layout()
    plt.show()


if __name__ == "__main__":
    user_input = int(input())
    if user_input == 0:
        run_benchmarks()
    else:
        solve(user_input, silent=False)
```